CodeGuard AI — Interview Questions & Answers

Predicted questions on this project, with model answers you can defend

How to use: read each Q, say your answer *out loud* before reading ours, then compare. Answers are true for *this* project — the numbers all come from your measured artifacts.

Red boxes = hostile / trap questions. Yellow boxes = numbers to memorize. Cross-read with *CodeGuardAI_Presentation_Mastery_Guide.pdf*.

A. Core concepts & approach

Q: What is this project in one sentence?

A: A CPU-only deep transfer learning system that flags vulnerable C functions: a frozen pretrained CodeBERT encoder produces embeddings, a small trained neural head classifies them, and the whole study (three experiments, selection, threshold, one-time test evaluation) follows a pre-registered protocol on the official Devign dataset.

Q: Why transfer learning instead of training a model from scratch?

A: CodeBERT is pretrained on 2.3M code/docstring pairs, so it already understands code syntax and semantics. Reusing it means a small head (~0.5M parameters) converges in seconds on the task with far less data than a from-scratch model would need — and it fits the CPU-only constraint.

Q: Is this fine-tuning? Did you fine-tune CodeBERT?

A: No. The 125M encoder weights are **frozen** — gradients never touch them; I run it under `torch.no_grad()` and only the classification head trains. Fine-tuning would update the encoder itself, which needs GPU-days. I describe it as "frozen CodeBERT features + a trained neural head", i.e., feature-extraction transfer learning.

Trap: if you say "fine-tuned", you've misdescribed your own project. Correct yourself immediately if you slip.

Q: If the encoder is frozen, is this even "deep learning"? Isn't it just a classifier?

A: It's deep transfer learning: the *features* are computed by a 12-layer transformer (deep network), and the head itself is a deep MLP (LayerNorm, 768→512→128→2 with GELU and dropout 0.2). What I train is shallower than end-to-end fine-tuning — that's a deliberate resource trade-off, not the absence of deep learning. A linear probe on the same features is my baseline, and the MLP beats it (AUC 0.657 vs 0.637), which shows the trained non-linearity adds real value.

Q: Why CodeBERT and not a plain NLP model like BERT or LSTM?

A: BERT is pretrained on natural language; CodeBERT is pretrained specifically on code plus natural language (bimodal), so its embeddings are far more meaningful for C functions. I also report published Devign reference points for BiLSTM/TextCNN/RoBERTa (59–61% accuracy) — the code-pretrained transformer is the right family. LSTMs would need to learn code structure from scratch on 22k samples.

Q: Describe the architecture end to end.

A: C source → CodeBERT BPE tokenizer (max 400 tokens, truncation) → frozen CodeBERT encoder → last hidden state (400×768) → masked mean pooling (padding excluded) → standardization with stored mean/std → MLP head (LayerNorm, 768→512→128→2, GELU, dropout 0.2) → softmax → P(vulnerable); decision at the frozen threshold 0.33.

Q: What is masked mean pooling and why not use [CLS]?

A: I average the token vectors over real (non-padding) positions: sum(mask·h)/sum(mask). CodeBERT was not pre-trained with a next-sentence objective, so its [CLS] token is a weak sentence representation for this task; masked mean pooling aggregates evidence across all tokens and is robust to variable-length input. It's also cheap — no extra parameters.

Q: Why a 400-token limit? Isn't that arbitrary?

A: Not arbitrary — measured. I benchmarked encoder throughput: at 400 tokens the model runs 3.91 samples/s, so the entire study (extraction + training + evaluation) finishes in ~85 minutes on my CPU. Longer sequences would double runtime while the vulnerability signal is usually local. The cost is truncation: 55.7% of train functions exceed 400 tokens, which I report as a limitation, not hide.

Q: What does the model's output actually mean?

A: A softmax score P(vulnerable) — an **uncalibrated** model score, not a probability of exploitation. I threshold it at 0.33 (chosen on validation) to produce a binary decision. The UI and docs explicitly call it advisory output, not a security guarantee.

B. Data

Q: Which dataset did you use and what are the splits?

A: Devign (function-level C vulnerabilities from FFmpeg and Qemu), via CodeXGLUE, with the official splits untouched: 21,854 train / 2,732 validation / 2,732 test. Labels: 0 = non-vulnerable, 1 = vulnerable. Test set is 45.94% vulnerable (1,255) / 54.06% safe (1,477).

Q: How do you know there's no data leakage?

A: Three layers: (1) I ran an audit that hashes function contents and checks cross-split duplicates — found 8 train∩validation, 4 train∩test, 1 validation∩test overlaps out of thousands, reported honestly; (2) model selection, threshold and ensemble decisions use validation only, and everything is frozen into a hashed bundle *before* the test set is opened; (3) the evaluation writes a `test_opened.json` signature, so re-running just re-displays stored results — development commands refuse after that.

Q: The dataset is almost balanced — why not just report accuracy?

A: 54/46 is mild imbalance, but accuracy hides the error structure: a model that misses most vulnerabilities can still look "accurate". The pre-registered objective was F1 (then recall), because for security screening the cost of a missed vulnerability (FN) is far higher than a false alarm. I report accuracy too — including the uncomfortable fact that at the frozen threshold my accuracy (53.2%) is below the majority guess (54.1%), and I explain why instead of hiding it.

Q: What did the audit find? Any duplicates?

A: Cross-split overlap is tiny (8/4/1 functions) — I left the official splits untouched rather than silently modifying them, and documented the numbers. Within-split duplicates and label distribution, class balance, function length and truncation statistics are all in `dataset_audit.json` and the EDA plot.

Q: What about label noise?

A: Devign labels are derived from commit history at function level, so they're noisy: a function can be marked vulnerable because of context elsewhere, and fixes can touch multiple functions. This caps achievable accuracy — it's one reason published results cluster in the low 60s, and a reason I don't chase higher numbers by tuning on the test set.

Q: What preprocessing did you do?

A: Minimal and deterministic: CodeBERT BPE tokenization, max length 400, padding/truncation per batch, attention mask for pooling. No stemming, no custom vocab, no oversampling in the main study (class weighting was experiment C2). Extraction is cached to disk so training reads tensors instead of re-running the encoder.

C. Training & experiments

Q: What experiments did you run?

A: Three, all on the same frozen features: C0 linear probe (768→2, CE loss) as baseline; C1 MLP head [512,128] with CE loss (the main model); C2 same MLP with class-weighted CE. Same seed 123456, batch 128, lr 0.001, weight decay 0.01, up to 24 epochs with early stopping patience 5 on validation loss.

Q: How did you choose the final model? What was the rule?

A: A rule fixed in code before running: rank by **validation ROC-AUC**, tie-break by tuned F1, then by recall. Results: mlp 0.6569 > weighted 0.6554 > linear 0.6372 → MLP selected. I never picked by test performance or by eye.

Q: Why is your threshold 0.33 and not 0.5?

A: Because 0.50 is a convention, not a decision-theoretic optimum. I searched the grid 0.05–0.95 (step 0.01) on validation, maximizing F1 with tie-breaks toward higher recall / fewer false negatives / closest to 0.50. The 0.33 cut-off reflects that missing a vulnerability costs more than a false alarm. Crucially, this was frozen before test evaluation.

Q: Did you try an ensemble? Why isn't it in the final model?

A: Yes — pre-registered rule: average probabilities of the top two models (MLP + weighted-MLP) and keep it only if it beats the single best tuned F1 on validation. It didn't, so it was rejected. The bundle records `ensemble: null`. I report this negative result rather than forcing a fancier model.

Q: How long did training take? Why is it so fast?

A: The expensive part is the one-time encoder pass (75 min for train+validation at 3.91 samples/s). Head training is ~20–25 seconds per experiment because there are only ~0.5M trainable parameters operating on cached 768-d features. The full study — extraction, three trainings, comparison, freeze, test evaluation, notebooks — took about 85 minutes on CPU.

Q: How do you prevent overfitting?

A: Several mechanisms: early stopping on validation loss (patience 5 — all three models stopped before epoch 10 of 24), weight decay 0.01, dropout 0.2 in the head, a small head relative to data, and — most importantly — an untouched test set plus a validation-based selection rule, so any overfitting to validation would show up as a test drop. Test AUC (0.6560) matches validation AUC (0.6569), so there's no memorization of the dev set.

Q: What is standardization and why did you add it?

A: The head input features get normalized with mean/std computed during training and stored as buffers inside the checkpoint (`set_statistics`), so inference applies the exact same transform without ever touching training data. It stabilizes training and removes a class of train/serve skew bugs. It's a documented decision (ADR-008).

Q: Could you reproduce these results?

A: Yes — fixed seed 123456, deterministic CPU kernels, cached features with fingerprints, and a single command chain from prepare → evaluate. The bundle is hash-verified at load. The two models differ only by which head was selected; running the pipeline again on the same machine reproduces the same metrics from the stored CSV/JSON.

D. Results & metrics

Numbers you must know cold:

Selected test F1 / baseline F1 / majority F1	**0.6470 / 0.5050 / 0.000**
Selected recall / baseline recall	**0.9331 / 0.4598** (TP 1171 vs 577)
Missed vulnerabilities (FN)	**84 vs 678** (−594)
ROC-AUC selected / baseline / majority	**0.6560 / 0.6206 / 0.500**
Accuracy selected / baseline / majority	0.5322 / 0.5860 / 0.5406
Validation AUCs (linear/mlp/weighted)	0.6372 / 0.6569 / 0.6554
Threshold (frozen) / bundle	0.33 / 8f0a21195194fc1912b9

Q: What are your headline results?

A: On the test set, my selected model scores F1 0.647 with 93.3% recall and ROC-AUC 0.656, versus my linear baseline's F1 0.505 / AUC 0.621, and a majority guess's F1 0. In absolute terms: 1,171 of 1,255 vulnerable functions caught, only 84 missed — the baseline missed 678. Delta F1 over baseline: +0.142.

Q: Your accuracy (53.2%) is LOWER than just always guessing "safe" (54.1%). Isn't the model useless?

A: The threshold was frozen on validation by the pre-registered "maximize F1, then recall" rule before the test set was opened — accuracy was never the target. At that operating point the model wins every imbalance-aware metric: F1 0.647 vs 0 vs 0.505, recall 93% vs 0% vs 46%, ROC-AUC 0.656 vs 0.5 vs 0.621, and it misses 84 vulnerabilities instead of 1,255 (guessing "safe" catches zero) or 678 (baseline). A screening tool that flags more code to catch nearly everything is the intended trade, and the operating point can be moved without retraining. If I had optimized accuracy, that would have been test-driven tuning — which my protocol forbids.

This is the question they will actually ask. Deliver it calmly with the numbers, not defensively. Then offer the two-table imbalance view from your deck.

Q: Explain precision vs recall here — what's the trade you made?

A: Precision 0.495: about half of what the model flags is a false alarm (1,194 FP of 2,365 flagged). Recall 0.933: it catches 93% of real vulnerabilities. At threshold 0.50 the model would be more precise but miss far more bugs — that's the baseline's profile (precision 0.56, recall 0.46, 678 FN). For security triage, a false positive costs review time; a false negative costs a breach — so I take the high-recall side.

Q: What does ROC-AUC 0.656 mean, honestly?

A: If you pick one vulnerable and one safe test function at random, the model ranks the vulnerable one higher about 65.6% of the time (random = 50%). It's a modest but real discrimination signal — better than baseline (0.621) and far above trivial (0.5), and consistent with validation (0.657). I describe it as a weak-to-moderate classifier, not a solved problem.

Q: Published fine-tuned CodeBERT gets ~62% accuracy on Devign; you got 53%. Why is yours worse?

A: We're not measuring the same thing at the same operating point. Those numbers are accuracy at (effectively) default thresholds after full fine-tuning; my headline number is F1- and recall-optimized at threshold 0.33, which trades accuracy for detection. My baseline at threshold 0.50 — comparable to how accuracy is usually quoted — is 58.6%, in the neighborhood of published BiLSTM/TextCNN results, and my model beats it on every threshold-free and imbalance-aware metric. Beyond that: I use a **frozen** encoder (a deliberate CPU-only scope), and 55.7% of functions are truncated at 400 tokens. Full fine-tuning with a GPU is exactly my listed future work.

Q: Is the difference from your baseline statistically meaningful?

A: The margin is large in practical terms — F1 +0.142, false negatives cut by 594 out of 678, on the same 2,732 functions — and validation and test agree (AUC 0.657 vs 0.656). I didn't run a formal significance test (e.g., McNemar or bootstrap CIs); if pressed, that's an honest gap I'd add — paired bootstrap over the test predictions would be the standard approach, and it can be done from the stored prediction files without re-training.

Q: Why is your model's threshold so low — is the model under-confident?

A: Because softmax outputs from a head trained with cross-entropy on frozen features aren't calibrated, and the class costs are asymmetric. The grid search simply found that F1 peaks at 0.33 on validation. Calibration (e.g., temperature scaling) is in my future-work list; the deployed rule is score → threshold → decision, with the threshold documented in the bundle.

E. Protocol, integrity & research rigor

Q: How do I know you didn't tune on the test set?

A: Structural guarantees, not promises: the threshold grid, selection rule and ensemble rule are implemented in code and fixed before evaluation; model + threshold + preprocessing are frozen into a hash-verified bundle by `finalize`; only then does `evaluate` run, writing `test_opened.json`. After that, development commands refuse to run on this study, and re-evaluation just re-plays stored results. All reported numbers come from files under `artifacts/cpu_v3/results/` — you can diff them.

Q: What if you could rerun the study — what would you change?

A: On protocol: add a second seed and report variance, plus bootstrap confidence intervals. On model: address truncation with windowing or a longer-context encoder, calibrate probabilities, and with a GPU, fine-tune the encoder end-to-end as an explicitly separate experiment. I'd also validate explanations against real CVE patches if line-level labels were available.

Q: Why did you reject the ensemble? Isn't ensembling always better?

A: Not always — it's better only if validation says so. Rule: keep the ensemble only if its tuned F1 beats the single best member's. Here it didn't, so keeping it would have added complexity for no measured gain, and possibly overfit my validation set. The rejection is recorded in `final_config.json` (`ensemble: null`) and shown in the deck.

Q: What is the difference between your smoke tests and real results?

A: Quick mode uses a tiny subset (e.g., 8 samples, 128 tokens) in a separate namespace (`artifacts/cpu_v3/smoke/`) to verify the pipeline runs end to end. Those numbers are never reported as findings — only the full official-split run counts. The UI also badges the smoke model so it can't be mistaken for the research bundle.

Q: What's in the model bundle? How do you know it wasn't corrupted?

A: Encoder weights, tokenizer, selected head (+ optional members), model specs with hyperparameters and data fingerprints, and a manifest. Every file has a SHA-256 hash recorded at finalization; `verify_bundle` re-hashes everything at load and refuses to run on any mismatch. `final_config.json` carries the selection rule, threshold rule, tuned thresholds and the bundle digest (8f0a21195194fc1912b9).

F. Explainability

Q: How does your explanation method work?

A: Token occlusion: for each token position (subsampled to a fixed budget), I replace it with the mask token, re-run the model, and measure the change in P(vulnerable). The signed difference is the token's influence on the predicted class; influences are aggregated per source line and the top lines are displayed with direction (supports/opposes the prediction).

Q: Does your heatmap show WHERE the vulnerability is?

A: No — and I never claim it does. It shows which tokens *influenced this model's decision*. The dataset has no line-level ground truth, so localization accuracy can't be measured, and a token can influence a prediction without being the bug. The UI states this explicitly: influence explanation, not a guaranteed vulnerability location.

Q: Why occlusion instead of attention maps or SHAP?

A: Occlusion is model-agnostic, easy to reason about (remove token → score change), and works directly with my batching on CPU. Attention weights aren't reliable explanations of contribution, and SHAP over 400 token positions would multiply an already expensive forward pass. I'd frame occlusion as the pragmatic, honest choice — with the caveat that it's approximated by mask-token substitution rather than re-tokenization.

G. Engineering & tools

Q: You had no GPU — what engineering did that force?

A: Frozen-encoder design so the expensive pass happens once (features cached to disk, resumable extraction); benchmarking to pick 400 tokens (3.91 samples/s); batched encoder inference; small heads trained on cached tensors in seconds; threshold grid on CPU-cheap validation logits; and a one-time test evaluation so we don't pay the encoder cost twice.

Q: What's the tech stack?

A: Python 3.12, PyTorch (CPU), HuggingFace Transformers (CodeBERT), Gradio for the web UI, NumPy/SciPy + matplotlib for metrics and plots, stdlib argparse CLI, pytest + ruff for tests and lint (24 tests passing, lint clean), TOML config. No cloud, no database, no external API — everything runs offline on localhost.

Q: How does the web demo work?

A: `codeguard demo` loads the verified bundle once, then serves a Gradio app on `127.0.0.1:7860` (auto-falls back to the next free port). Paste C code (or click an example) → it runs tokenizer → encoder → head → threshold, optionally the occlusion explanation, and shows a verdict badge, metrics, line-influence table and raw JSON. Input validation covers empty, huge (>200k chars) and unexpected errors — friendly panels, no tracebacks in the browser.

Q: Why Gradio? Why not a real web app?

A: Gradio gives a clean, interactive UI in ~100 lines with zero frontend build, which fits a local, offline, CPU-only demo. The product of this project is the study + model, not a web platform; the demo exists to make predictions tangible for review.

Q: How do you handle errors or bad input in production terms?

A: Validate at the boundary (empty/oversized/non-string input rejected with clear messages), catch-all in the handler with traceback to the server console (never leaked to the UI), hash verification before loading weights, queue concurrency limit to prevent overlapping inference, and explicit state banners (smoke model vs research bundle, truncation notices).

H. Limits, future work & big-picture

Q: What are the biggest weaknesses of your project?

A: Three: (1) truncation — over half of train functions exceed 400 tokens; (2) frozen encoder caps accuracy below full fine-tuning, and precision at the chosen threshold is 0.495, so about half the flags are false alarms; (3) function-level noisy labels and no line-level ground truth, which limits both achievable accuracy and explanation validation. All are documented in STATUS/DECISIONS, not buried.

Q: What would you do with a GPU and more time?

A: Fine-tune CodeBERT end-to-end (the experiment my protocol explicitly excludes), sweep hyperparameters on validation, calibrate probabilities, try sliding-window handling of long functions, evaluate code-specific encoders, and validate explanations against real CVE patches. The pipeline already supports optional LoRA as a middle ground.

Q: Where would you deploy this and how would it fit a security workflow?

A: As a CI/pre-commit screening step: run on changed functions, surface high-recall flags for human review, keep humans in the loop for verdicts (precision isn't high enough for blocking merges). Threshold can be tuned per context without retraining, and explanations give the reviewer a starting point.

Q: How would this generalize beyond FFmpeg/Qemu?

A: I'd validate on other C projects (cross-project evaluation), check for style/tokenizer distribution shift, and ideally retrain with augmented data. The current model should be treated as in-distribution — I make no cross-project claim.

I. Behavioral & high-pressure questions

Q: Why should I believe your numbers?

A: Because they're machine-generated into CSV/JSON artifacts you can open (`validation_comparison.csv`, `test_comparison.json`), the model/threshold were frozen with SHA-256 hashes before a single test metric existed, the test set was opened exactly once with a recorded signature, and the entire run is reproducible in ~85 minutes on this CPU. You're welcome to re-run `codeguard evaluate` — it will only re-display the stored results.

Q: This looks like a class project, not research. What's novel?

A: The novelty claim is deliberately modest — I don't claim SOTA. The value is a rigorous, fully audited transfer-learning study under a hard CPU constraint: pre-registered selection/threshold/ensemble rules enforced in code, hash-verified freezing, one-time test evaluation, honest negative results (rejected ensemble), and an honest treatment of the accuracy-vs-F1 trade. Reproducibility and protocol discipline are the contribution; the modeling choices (frozen CodeBERT + trained head) are standard practice made transparent.

Q: What broke during the project? How did you fix it?

A: Examples: the threshold floor of 0.20 in an early version hid better low-threshold solutions — I widened the grid to 0.05–0.95 and added regression tests; ensembles in quick-smoke mode produced identical members — I made rejection explicit; the demo originally crashed when the port was already in use and surfaced raw tracebacks — I added port fallback, friendly error panels and a test suite for the formatting/error paths. Pattern: reproduce, fix at the root, lock it with a test, document the decision.

Q: If we gave you a GPU tomorrow, first thing you'd do?

A: Run an end-to-end fine-tuning experiment as a *new study* (never touching the frozen cpu_v3), comparing AUC/F1 against the frozen-encoder result under the identical selection protocol — plus truncation handling for long functions. That's a clean before/after on the single biggest lever.

Q: Do you know what a false negative means in security terms?

A: A vulnerable function the model called safe — a potential undetected breach path. That's why my selection rule optimizes F1 with recall tie-breaks and why the frozen threshold cuts FN from 678 (baseline) to 84. I'd rather review extra code than ship a known hole.

Q: What would you do differently if you started over?

A: Add confidence intervals and a second seed from day one; plan long-function handling earlier (windowing) instead of accepting 55.7% truncation; write the interview deck generator and artifacts documentation alongside the pipeline rather than at the end; and instrument a "results provenance" check that traces every reported number to its file automatically.

J. Quick-fire round (one-line answers)

Question	One-liner
Two classes — which is positive?	1 = vulnerable (binary P/R/F1 all refer to class 1)
Encoder params / trainable params?	125M frozen / ~0.5M in the head
Random seed?	123456 (fixed, recorded in every model spec)
Loss functions used?	cross-entropy (C0, C1) and class-weighted CE (C2)

Early stopping signal / patience?	validation loss, patience 5 (stopped at epochs 9/10/10)
Selection metric?	validation ROC-AUC → tuned F1 → recall
Threshold search?	0.05–0.95 step 0.01, F1-first, frozen at 0.33
Ensemble used in final model?	No — rejected by validation rule
How many times was test opened?	Exactly once (signature in test_opened.json)
Truncation rate?	55.7% of train functions > 400 tokens
Encoder throughput?	3.91 samples/s @400 tokens
Total study runtime?	~85 minutes, CPU only
Tests / lint?	24 passing / ruff clean
Explainability method?	mask-token occlusion → per-line signed influence
UI framework / port?	Gradio, 127.0.0.1:7860 (auto port fallback)
Dataset / splits?	Devign — 21,854 / 2,732 / 2,732
Final bundle ID?	8f0a21195194fc1912b9 (SHA-256 verified)

Three things to say unprompted (they signal mastery):

- "Selection, threshold and ensemble rules were fixed in code *before* evaluation — the test set was opened once, and I can show you the signature."
- "At threshold 0.50 my model's accuracy is 58.6% and it beats the baseline at every threshold-free metric; at the frozen 0.33 I trade precision for recall by design — F1 0.647, 84 missed vulnerabilities instead of 678."
- "This is **frozen** CodeBERT transfer learning, never fine-tuning — the whole study is reproducible on a CPU in about 85 minutes."

All numbers are measured artifacts from `artifacts/cpu_v3/` (2026-10-01). Published literature figures require exact citations before you quote them.